LAB03B - For loops

October 26, 2026

[2]:
```
# In Python it is standard practice to import the modules we need at the very␣
 ↪top of our scripts
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
```

1 'For' loops

Now that we will be processing larger numbers of datasets, we want to make sure we are not writing repetitive code. A `for` loop is great for automating things that we are going to do over and over again. Consider these examples, which you will find helpful for this (and later) exercises.

[3]:
```
print('We are about to loop over the following numbers:')
print(np.arange(10))
for n in np.arange(10):
    # The code that is indented here will be executed ten times.
    # In each 'iteration' of the loop, the variable n will have a different␣
 ↪value
    print(' We are now executing the code in the loop, with n = {0}'.format(n))
print('The loop has finished executing')
```

```
We are about to loop over the following numbers:
[0 1 2 3 4 5 6 7 8 9]
 We are now executing the code in the loop, with n = 0
 We are now executing the code in the loop, with n = 1
 We are now executing the code in the loop, with n = 2
 We are now executing the code in the loop, with n = 3
 We are now executing the code in the loop, with n = 4
 We are now executing the code in the loop, with n = 5
 We are now executing the code in the loop, with n = 6
 We are now executing the code in the loop, with n = 7
 We are now executing the code in the loop, with n = 8
 We are now executing the code in the loop, with n = 9
The loop has finished executing
```

[4]:
```
print('We are about to loop over the following numbers:')
print(np.arange(0, 1, 0.2))
for n in np.arange(0, 1, 0.2):
```

```
    print(' We are now executing the code in the loop, with n = {0:.1f}'.
  ↪format(n))
print('The loop has finished executing')
```

```
We are about to loop over the following numbers:
[0.  0.2 0.4 0.6 0.8]
 We are now executing the code in the loop, with n = 0.0
 We are now executing the code in the loop, with n = 0.2
 We are now executing the code in the loop, with n = 0.4
 We are now executing the code in the loop, with n = 0.6
 We are now executing the code in the loop, with n = 0.8
The loop has finished executing
```

[5]:
```
# We can loop over a list of strings, as well as numbers
# (remember this: it will be useful later!)
print('We are about to loop over the following strings:')
print(['file1.csv', 'file2.csv', 'file3.csv'])

# Note that we have made a 'list' of strings by enclosing them in square␣
  ↪brackets.
for filename in ['file1.csv', 'file2.csv', 'file3.csv']:
    print(' We are now executing the code in the loop, with filename = {0}'.
  ↪format(filename))
print('The loop has finished executing')
```

```
We are about to loop over the following strings:
['file1.csv', 'file2.csv', 'file3.csv']
 We are now executing the code in the loop, with filename = file1.csv
 We are now executing the code in the loop, with filename = file2.csv
 We are now executing the code in the loop, with filename = file3.csv
The loop has finished executing
```

2 Exercise : 'For' loops (5 Marks)

1) Load the data file `gaussian-data.csv`, which contains five datasets, each assumed to be a (small) set of normal (or Gaussian) distributed data. Print out the value of the variable `myArray.shape` (replace `myarray` with the name of your own variable) to see the actual shape of the array you loaded.

2) Use a `for` loop to calculate and print the mean, standard deviation and standard error for each of the 5 sets of data in turn. *Hint: you may find it helpful to use* ***`myArray.shape`*** *as part of your code for the* ***`for`*** *loop.*

3) Are the 5 sets of data consistent with each other, in the statistical sense (add a comment in the Markdown cell)? Remember, consistent datasets are usually assumed to differ by no more than 3 standard deviations.

[ ]:
```
# Put your answer to this exercise here
```

Are the datasets consistent with each other? Write your answer here

[ ]: